

Arquitectura del sistema

Tipo de aplicación

La solución propuesta corresponde a una aplicación móvil nativa para Android, desarrollada utilizando Kotlin y Jetpack Compose como framework principal de interfaz de usuario

La elección de una aplicación nativa se fundamenta en la necesidad de ofrecer una experiencia de usuario fluida, responsiva y alineada con las guías de diseño del sistema operativo Android. A diferencia de enfoques híbridos, el desarrollo nativo permite un mejor aprovechamiento de los recursos del dispositivo, especialmente en escenarios donde se requiere renderizado eficiente de listas dinámicas, manejo de imágenes y navegación entre pantallas

Adicionalmente, el enfoque nativo facilita la integración con servicios del ecosistema Android, como Firebase Cloud Messaging (FCM), lo cual resulta relevante considerando posibles extensiones futuras de la aplicación, como notificaciones push. En este contexto, la decisión contribuye tanto a la calidad de la experiencia de usuario como a la escalabilidad del sistema

Arquitectura del front-end

La aplicación se basa en el patrón arquitectónico MVVM (Model–View–ViewModel), el cual permite estructurar el sistema de manera clara, separando las responsabilidades entre las distintas capas

La arquitectura sigue el siguiente flujo lógico:

UI (Compose Screens) → ViewModel → Repository → API Service (Mock)

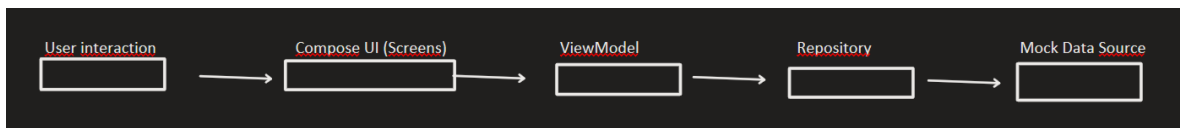
En esta estructura, la capa de interfaz de usuario está compuesta por pantallas desarrolladas con Jetpack Compose, las cuales se encargan exclusivamente de renderizar el estado y reaccionar a cambios en los datos. Estas pantallas no contienen lógica de negocio, lo que permite mantener un código limpio y fácil de mantener

El ViewModel actúa como intermediario entre la UI y las capas de datos. Su responsabilidad principal es gestionar el estado de la interfaz, procesar interacciones del usuario y preparar los datos en un formato adecuado para su visualización. Además, permite desacoplar completamente la lógica de presentación de la implementación visual

El Repository abstrae el acceso a los datos, proporcionando una interfaz limpia para el ViewModel. En el contexto de esta prueba de concepto, el Repository utiliza datos simulados, pero su diseño permite reemplazar fácilmente esta fuente por un servicio real en el futuro

El API Service representa una capa conceptual que simula las llamadas a un servicio REST, alineándose con una posible integración futura con un backend real

Diagrama de arquitectura del sistema



Flujo general del sistema

El flujo del sistema describe cómo interactúan los diferentes componentes desde que el usuario realiza una acción hasta que se presenta la información en pantalla

Cuando el usuario accede a la aplicación, la interfaz de usuario solicita datos al ViewModel. Este, al mismo tiempo, delega la obtención de la información al Repository, el cual retorna datos simulados que representan la respuesta de un servicio externo. Una vez recibidos los datos, el ViewModel actualiza su estado interno, lo que provoca automáticamente la recomposición de la interfaz en Jetpack Compose y la actualización de los elementos visuales

En el caso específico de la visualización de espacios de coworking, el usuario accede a una pantalla de listado donde se muestran múltiples espacios disponibles. El ViewModel solicita la lista al Repository mediante un método como: `getSpaces()`

El Repository retorna una colección de objetos que incluyen atributos como nombre, imagen, precio y disponibilidad. Estos datos son observados por la UI, que los presenta utilizando componentes como `LazyColumn` y `Card`

Cuando el usuario selecciona un espacio, se activa la navegación hacia la pantalla de detalle, pasando como parámetro el identificador o el objeto completo. El ViewModel correspondiente procesa esta información y expone los datos necesarios para que la UI renderice la vista detallada del espacio seleccionado